Experiment – 8

Name: Manan Verma

UID: 24BAI70008

Section: 24AIT_KRG G1

A. Experimental Procedure

Step 1: Implementation

Binary Search,

Subset Sum (Verification and Decision versions),

Traveling Salesman Problem (Brute Force).

The implementation should ensure correctness, proper handling of edge cases, and readability.

Step 1: Implementation

The implementation phase focused on developing three distinct algorithms to observe their behavior across different complexity classes. Each was coded to ensure high readability and robust handling of edge cases.

Binary Search (O(log n)):

- Approach: Implemented using an iterative divide-and-conquer strategy on a pre-sorted array.
- Logic: The algorithm repeatedly identifies the middle element and discards the half of the array that cannot contain the target.
- Edge Cases: Handled scenarios involving empty arrays, single-element arrays, and targets that were either smaller than the minimum or larger than the maximum element.

Subset Sum (Verification and Decision):

- Decision Version (O(2^n)): Developed using recursion with memoization to determine if any combination of numbers in a set adds up to a target value K.

- Verification Version (O(n)): Implemented a simple linear-time function that takes a “candidate subset” and validates if its sum equals K.
- Edge Cases: Addressed inputs where the target K is zero, cases with negative integers (if applicable), and sets where the sum of all elements is still less than the target.

Traveling Salesman Problem (Brute Force - O(n!)):

- Approach: Utilized a permutation-based strategy to calculate the total distance of every possible Hamiltonian cycle in a graph.
- Logic: The implementation iterates through all (n-1)! possible paths, keeping track of the one with the minimum total weight.
- Edge Cases: Handled small graphs (2–3 nodes), disconnected graphs, and weighted edges to ensure the logic remained sound regardless of graph density.

Step 2: Measure Metrics

Record key performance metrics for each execution, including execution time, approximate number of operations, and feasibility status (Completed or Timeout). Maintain a structured log for further analysis.

To analyze the “Factorial Wall” and “Exponential Growth,” performance metrics were logged during execution.

- **Execution Time:** Recorded using high-precision timers (milliseconds) to capture the difference between sub-millisecond search results and multi-second brute-force calculations.
- **Approximate Number of Operations:** Calculated based on the Big O theoretical growth (n! for TSP, 2^n for Subset Sum, and log n for Binary Search) to correlate theoretical complexity with actual CPU work.
- **Feasibility Status: Completed:** Assigned to executions where the algorithm finished within a 30-second window.
- **Timeout:** Assigned to instances where the combinatorial explosion made it impossible for the computer to finish (typically seen in TSP with n > 15).

B. Analysis Tasks (Critical Thinking)

Q1. Why does Binary Search consistently demonstrate efficient performance across large input sizes?

Binary Search stays fast even when the input size becomes very large because it doesn't check every element one by one. Instead, it keeps cutting the search space into half after every comparison.

So even if the size increases a lot, the number of steps doesn't increase much. For example, going from thousands to millions of elements only adds a few extra steps.

This is because its time complexity is logarithmic (O(log n)), which grows very slowly compared to linear methods. That's why it works efficiently even for large datasets.

Q2. Explain why Subset Sum is computationally difficult to solve but relatively easy to verify.

The Subset Sum problem is hard to solve because we may have to check all possible subsets to find the correct one. The total number of subsets grows exponentially (2^n), which becomes extremely large even for moderate values of n.

However, verifying a solution is much simpler. If someone gives us a subset, we just need to add its elements and check if it equals the target value. This only takes linear time.

So basically, finding the answer is difficult, but checking an already given answer is easy.

Q3. Identify the input size at which the Traveling Salesman Problem becomes infeasible and justify the reason.

For the brute-force approach of the Traveling Salesman Problem, it becomes impractical somewhere around 12 to 15 cities.

This happens because the number of possible routes grows factorially (n!), which increases extremely fast. Even a small increase in the number of cities leads to a huge jump in the number of possible paths.

After a certain point, the computation takes too long to finish within a reasonable time, making it infeasible.

Q4. Differentiate between solving a problem and verifying a given solution with appropriate examples.

Solving a problem means actually finding the answer from scratch by exploring all possibilities or using an algorithm.

For example, in Subset Sum, finding a subset whose sum equals a target value is solving the problem.

Verifying a solution means checking whether a given answer is correct or not.

For example, if a subset is already given, we just add its elements and see if it matches the target.

So solving requires searching, while verifying only requires checking.

Q5. Discuss why NP-Complete problems are considered the most challenging class within NP.

NP-Complete problems are considered very difficult because no efficient (polynomial-time) solution is known for them.

They usually involve checking a very large number of possibilities, which grows exponentially with input size.

Another important reason is that if we somehow find an efficient solution for even one NP-Complete problem, then all problems in NP can be solved efficiently.

Because of this, they are seen as the hardest problems in the NP class. Examples include Subset Sum and Traveling Salesman Problem.